

Cypher 5 vs Cypher 25 : Comparaison sur des Données de Vols Américains

Romain Groult & Alban Talagrand

Janvier 2025

Introduction

Ce projet compare les performances et l’expressivité de Cypher 5 et Cypher 25 sur un dataset réel de vols. Le dataset choisi représente les vols américains de la première semaine de janvier 2015 : 107 230 vols entre 313 aéroports, opérés par 14 compagnies.

Choix et Modélisation des Données

Source des données

Choix initial : Dataset IDFM (abandonné) Nous avons initialement envisagé d’utiliser les données GTFS d’IDFM représentant le réseau de transports en commun francilien. Ce dataset présentait plusieurs fichiers interconnectés :

- `agency.csv` : Agences de transport
- `routes.csv` : Lignes de transport
- `trips.csv` : Trajets
- `stop_times.csv` : Horaires d’arrêt pour chaque trajet
- `stops.csv` : Stations et arrêts
- `transfers.csv` : Correspondances entre arrêts
- `pathways.csv` : Chemin piétons dans les stations

Problème identifié : Cette structure est orientée relationnelle. La modélisation en graphe était trop complexe :

- Les “nœuds” étaient des stations, mais les relations entre elles ne sont pas directes
- Il faut passer par 4 tables intermédiaires (route -> trip -> stop_time) pour relier deux stations
- Les propriétés intéressantes (horaires, fréquences) sont dispersées dans plusieurs tables

C’est d’ailleurs un cas où PostgreSQL est mieux adapté que Neo4j : les données GTFS ont été conçues pour des requêtes relationnelles (jointures, agrégations temporelles).

Choix final : Dataset de vols américains Dataset Kaggle “2015 Flight Delays and Cancellations” (US Department of Transportation) :

- Source : 5,8 millions de vols sur l’année 2015
- Échantillon retenu : Première semaine de janvier

- Raison : Volume raisonnable tout en conservant un nombre de données suffisantes pour avoir des requêtes intéressantes



FIGURE 1 – Flight Data Visualization

Visualisation du dataset

Nettoyage des données

Le script `scripts/normalize_data.py` effectue plusieurs transformations :

1. **Filtrage temporel** : Sélection des vols du 1er au 7 janvier 2015
2. **Gestion des timestamps** :

- Conversion des horaires HHMM vers ISO 8601
 - Traitement du minuit (2400 -> 0000)
 - Détection des vols de nuit (arrivée le lendemain)
3. **Suppression des valeurs manquantes** : Retrait des vols sans horaires de départ/arrivée
 4. **Filtrage des aéroports** : Seuls les aéroports utilisés dans les vols sont conservés

Modèle de graphe Neo4j

Nœuds :

- **Airport** (313) : Code IATA, nom, ville, état, coordonnées GPS
- **Airline** (14) : Code IATA, nom de la compagnie

Relations :

- **FLIGHT** (107 230) : Propriétés :
 - **departure_ts** / **arrival_ts** : Timestamps ISO 8601
 - **distance** : Distance en miles
 - **delay** : Retard au départ en minutes (peut être négatif)
 - **airline** : Code de la compagnie

Contraintes :

- Unicité des codes IATA (airports et airlines)
- Index sur **departure_ts**, **distance**, **delay** pour optimiser les requêtes

Choix de modélisation : Chaque vol est une relation unique, identifiée par son timestamp. Cela évite le problème des multi-edges : ANC->SEA à 23h54 et ANC->SEA à 08h30 sont deux relations distinctes, ce qui simplifie les requêtes de chemins temporels.

Modèle relationnel PostgreSQL

airlines

- **iata_code** (PK)
- **name**

airports

- **iata_code** (PK)
- **name, city, state, country**
- **latitude, longitude**

flights

- **id** (PK)
- **source** (FK -> airports.iata_code)
- **target** (FK -> airports.iata_code)
- **airline** (FK -> airlines.iata_code)
- **departure_ts, arrival_ts**
- **distance, delay**
- **CHECK** (source != target AND distance > 0)

Index sur **source**, **target**, **departure_ts** pour optimiser les requêtes récursives.

Comparaison des Requêtes

1. Chemins avec propriété croissante

Problème : Trouver des itinéraires LAX->JFK où le retard augmente à chaque escale.

Cypher 5 (NOT EXISTS) :

```
MATCH path = (start:Airport {iata_code: 'LAX'})
  -[:FLIGHT*2..4]->(end:Airport {iata_code: 'JFK'})
WHERE NOT EXISTS {
  WITH path
  UNWIND range(0, size(relationships(path))-2) AS i
  WITH relationships(path) AS rels, i
  WHERE rels[i].delay >= rels[i+1].delay
  RETURN 1
}
RETURN [n IN nodes(path) | n.iata_code] AS route
LIMIT 50;
```

Stratégie : Génère TOUS les chemins (319 631), puis filtre *a posteriori*.

Résultats :

- Temps : ~1,5 seconde
- DB Hits : 16 340 381
- Rows traitées : 320 799

Cypher 25 (allReduce) :

```
MATCH path = (start:Airport {iata_code: 'LAX'})
  ((:Airport)-[f:FLIGHT]->(:Airport)){2,4}
  (end:Airport {iata_code: 'JFK'})
WHERE allReduce(
  prev_delay = -999999.0,
  rel IN f |
  CASE
    WHEN rel.delay > prev_delay THEN rel.delay
    ELSE null
  END,
  prev_delay IS NOT NULL
)
RETURN path LIMIT 50;
```

Stratégie : Filtre PENDANT la traversée grâce à l'opérateur `Repeat(Trail)`.

Résultats :

- Temps : ~3,5 ms
- DB Hits : 39 904
- Rows traitées : 9 290

Gain :

- Temps : 1 496,5ms de moins

- DB Hits : 16 300 477 de moins
- Rows traitées : 311 509 de moins

Plan d'exécution :

- Cypher 5 : VarLengthExpand génère tout puis Apply+Anti filtre
- Cypher 25 : Repeat(Into,Trail) avec pruning inline

2. Quantified Graph Patterns

Problème : Trouver des itinéraires avec exactement N escales, sans repasser par le même aéroport.

Cypher 5 (explicite) :

```
MATCH path = (start:Airport {iata_code: 'LAX'})
  -[:FLIGHT]->(hub1:Airport)
  -[:FLIGHT]->(hub2:Airport)
  -[:FLIGHT]->(end:Airport {iata_code: 'JFK'})
WHERE start <> hub1 AND start <> hub2 AND start <> end
  AND hub1 <> hub2 AND hub1 <> end
  AND hub2 <> end
RETURN [n IN nodes(path) | n.iata_code] AS route
LIMIT 10;
```

7 lignes, 6 comparaisons pour éviter les cycles.

Cypher 25 (quantified) :

```
MATCH path = (start:Airport {iata_code: 'LAX'})
  (()-[:FLIGHT]->(:Airport)){3}
  (end:Airport {iata_code: 'JFK'})
WHERE allReduce(
  seen = [],
  n IN nodes(path) | seen + n,
  single(x IN seen WHERE x = n)
)
RETURN [n IN nodes(path) | n.iata_code] AS route
LIMIT 10;
```

1 ligne pour le pattern, allReduce() pour détecter les cycles. Pour un range (2-3 escales), on change juste {3} en {2,3}.

Intérêt : Améliore la maintenabilité. Modifier la profondeur = changer un chiffre vs réécrire le pattern entier.

3. Plus courts chemins pondérés

Problème : Trouver l'itinéraire JFK->DAY minimisant la distance totale.

C'est ici que les limites de Cypher pur deviennent flagrantes.

Cypher 5 (shortestPath) Pas pondéré :

```

MATCH path = shortestPath(
  (start:Airport {iata_code: 'JFK'})-[:FLIGHT*]->(end:Airport {iata_code: 'DAY'})
)
RETURN [n in nodes(path) | n.iata_code] AS route,
       reduce(dist = 0, r in relationships(path) | dist + r.distance) AS
  ↪ total_distance;

```

Résultat :

- Route : [JFK, ATL, DAY]
- Distance : 1192 miles
- Temps : 15 ms

Problème : `shortestPath()` minimise le nombre de sauts, pas la distance. Le vrai chemin optimal (JFK->BWI->DAY) fait 590 miles mais passe par 2 sauts aussi. L'algorithme BFS ne considère pas les poids.

Cypher 5/25 pondéré :

```

MATCH path = (start:Airport {iata_code: 'JFK'})-[:FLIGHT*1..3]->(end:Airport
  ↪ {iata_code: 'DAY'})
WITH path, reduce(dist = 0, r in relationships(path) | dist + r.distance) AS
  ↪ total_distance
ORDER BY total_distance
LIMIT 1
RETURN [n in nodes(path) | n.iata_code] AS route, total_distance;

```

Résultat :

- Route : [JFK, BWI, DAY]
- Distance : 590 miles (optimal)
- Temps : 293 ms

Explosion combinatoire :

- *1..2 (2 sauts max) : 293 ms
- *1..3 (3 sauts max) : 137 secondes
- *1..4 (4 sauts max) : timeout

Au-delà de 2-3 sauts, la requête devient inutilisable.

GDS Dijkstra :

```

CALL gds.graph.project('flights', 'Airport', {
  FLIGHT: { properties: 'distance' }
});

MATCH (source:Airport {iata_code: 'JFK'})
MATCH (target:Airport {iata_code: 'DAY'})
CALL gds.shortestPath.dijkstra.stream('flights', {
  sourceNode: source,
  targetNode: target,
  relationshipWeightProperty: 'distance'
})

```

```
YIELD totalCost, nodeIds
RETURN [nodeId in nodeIds | gds.util.asNode(nodeId).iata_code] AS route,
       totalCost AS total_distance;
```

Résultat :

- Route : [JFK, BWI, DAY]
- Distance : 590 miles (optimal garanti)
- Temps : 37 ms

Comparaison :

- Cypher 5 : Non pondéré, pas optimal
- Cypher 25 : Timeout au-delà de 2-3 sauts
- GDS Dijkstra : Toujours performant, optimal, scalable

Conclusion : Pour des chemins pondérés, GDS est obligatoire. Cypher pur n'a pas les structures de données nécessaires (priority queue).

4. Implémentation d'algorithmes GDS en Cypher 25

Objectif : Reproduire des algorithmes de GDS avec du Cypher pur pour comparer les approches.

Degree Centrality :

GDS :

```
CALL gds.degree.stream('my-graph')
YIELD nodeId, score
RETURN gds.util.asNode(nodeId).iata_code AS airport, score
ORDER BY score DESC LIMIT 10;
```

Cypher 25 :

```
MATCH (a:Airport)
OPTIONAL MATCH (a)-[:FLIGHT]->()
WITH a, count(*) AS out_degree
OPTIONAL MATCH (a)<-[:FLIGHT]-()
WITH a, out_degree, count(*) AS in_degree
RETURN a.iata_code AS airport, out_degree + in_degree AS degree
ORDER BY degree DESC LIMIT 10;
```

Résultat : Performances similaires pour ce cas simple.

Triangle Count :

GDS :

```
CALL gds.triangleCount.stream('my-graph')
YIELD nodeId, triangleCount
RETURN gds.util.asNode(nodeId).iata_code AS airport, triangleCount
ORDER BY triangleCount DESC LIMIT 10;
```

Cypher 25 :

```
MATCH (a:Airport)-[:FLIGHT]->(b:Airport)-[:FLIGHT]->(c:Airport)-[:FLIGHT]->(a)
RETURN a.iata_code AS airport, count(*) AS triangles
ORDER BY triangles DESC LIMIT 10;
```

Problème : Cette requête compte chaque triangle 3 fois (une fois par sommet). Correction :

```
WITH a, count(*) / 3 AS triangleCount
```

Observation : Pour des algorithmes polynomiaux simples, Cypher 25 est compétitif. L'overhead de GDS (projection du graphe) peut même le rendre plus lent.

Limite : Dès qu'on passe à des algorithmes nécessitant des structures de données avancées (Dijkstra, PageRank, Betweenness Centrality), GDS devient indispensable. Ces algorithmes ne sont pas implémentables efficacement en Cypher.

Équivalents SQL

Chemins avec propriété croissante

SQL ne supporte pas nativement les path queries. Il faut utiliser une requête récursive :

```
WITH RECURSIVE paths AS (
  -- Cas de base : vols directs depuis LAX
  SELECT
    source,
    target,
    ARRAY[source, target] AS route,
    ARRAY[delay] AS delays,
    1 AS hops,
    delay AS last_delay
  FROM flights
  WHERE source = 'LAX'

  UNION ALL

  -- Cas récursif : ajouter un vol
  SELECT
    f.source,
    f.target,
    p.route || f.target,
    p.delays || f.delay,
    p.hops + 1,
    f.delay
  FROM paths p
  JOIN flights f ON p.target = f.source
  WHERE f.delay > p.last_delay
    AND p.hops < 4
    AND NOT (f.target = ANY(p.route))
)
SELECT route, delays
FROM paths
```



```
WHERE target = 'JFK' AND hops >= 2
LIMIT 50;
```

Différences :

- Plus verbeux (25 lignes vs 15 pour Cypher 5, vs 10 pour Cypher 25)
- Nécessite de gérer manuellement les cycles (NOT (f.target = ANY(p.route)))
- Moins lisible : la logique métier (delay croissant) est noyée dans la syntaxe récursive

Performance : Comparable à Cypher 5 (~1-2 secondes). SQL n'a pas d'équivalent à `allReduce()` pour optimiser.

Plus courts chemins

SQL récursif avec BFS :

```
WITH RECURSIVE paths AS (
  SELECT
    source,
    target,
    ARRAY[source, target] AS route,
    distance,
    1 AS hops
  FROM flights
  WHERE source = 'JFK'

  UNION ALL

  SELECT
    f.source,
    f.target,
    p.route || f.target,
    p.distance + f.distance,
    p.hops + 1
  FROM paths p
  JOIN flights f ON p.target = f.source
  WHERE p.hops < 3
    AND NOT (f.target = ANY(p.route))
)
SELECT route, distance
FROM paths
WHERE target = 'DAY'
ORDER BY distance
LIMIT 1;
```

Problème : Même limitation que Cypher pur. Sans priority queue, impossible d'implémenter Dijkstra efficacement. La requête explore exhaustivement.

Performance : Comparable à Cypher exhaustif (centaines de ms pour 2 sauts, timeout à 3+).

Limitations et Extensions Possibles

Limites du projet

1. **Période courte** : 1 semaine de données limite la profondeur des chemins testables (max 2-3 escales réalistes dans notre graphe).
2. **Pas de contraintes temporelles strictes** : On n'a pas implémenté de requêtes vérifiant que l'arrivée d'un vol précède le départ du suivant (avec marge pour correspondance). C'est faisable avec `allReduce()` mais complexifie les requêtes.
3. **GDS vs SQL** : Pas de comparaison GDS vs requêtes PostgreSQL optimisées (ex : pgRouting pour Dijkstra). Aurait pu enrichir l'analyse.
4. **REPEATABLE ELEMENTS non testé** : Cypher 25 introduit la syntaxe **REPEATABLE ELEMENTS** qui permet aux chemins de revisiter les mêmes nœuds. Nous n'avons pas utilisé cette feature car elle n'était pas pertinente pour notre modèle de données :
 - Chaque vol (**FLIGHT**) est une relation unique avec un timestamp distinct
 - Exemple : ANC->SEA à 23h54 et ANC->SEA à 08h30 sont deux relations différentes
 - Sans **REPEATABLE ELEMENTS** : on ne peut pas revisiter le même nœud (aéroport)
 - Avec **REPEATABLE ELEMENTS** : on peut revisiter le même nœud (aéroport)
 - Dans les deux cas, on peut emprunter différentes relations entre les mêmes nœuds

Pour des itinéraires réalistes, les passagers ne font pas de circuits comme LAX->ATL->LAX->JFK. La feature aurait été utile avec des relations sans timestamps.

Conclusion

Ce projet confirme empiriquement les résultats théoriques de l'article SIGMOD :

1. **Cypher 5 avec `reduce()` dans **WHERE** ne scale pas** : 16M DB hits vs 40k pour la même requête en Cypher 25 (facteur 409x).
2. **`allReduce()` résout le problème** : En intégrant le filtre dans la traversée, Cypher 25 évite l'explosion combinatoire.
3. **Quantified patterns simplifient le code** : Réduction de code pour des patterns répétitifs.
4. **GDS est indispensable pour les chemins pondérés** : Cypher pur (même v25) n'a pas les structures de données pour Dijkstra. Cela timeout au-delà de 2-3 sauts.
5. **SQL n'est pas adapté aux path queries** : Plus verbeux, moins lisible, performances comparables à Cypher 5 (pas d'équivalent à `allReduce()`).